

Exercitii Structuri de Date

1. Care este complexitatea timp pentru accesarea unui element intr-un vector dinamic pe baza de index?
 - (a) $O(1)$
 - (b) $O(n)$
 - (c) $O(\log_2 n)$
 - (d) $O(n * \log_2 n)$
2. Care TAD foloseste principiul LIFO (Last In First Out)?
 - (a) stiva
 - (b) coada
 - (c) colectie
 - (d) multime
3. Care este complexitatea in cazul defavorabil pentru cautarea liniara?
 - (a) $O(1)$
 - (b) $O(n)$
 - (c) $O(\log_2 n)$
 - (d) $O(n * \log_2 n)$
4. Care dintre urmatoarele operatii este mai eficienta pentru vectorul dinamic?
 - (a) Inserarea la inceput
 - (b) Stergerea de la mijloc
 - (c) Accesarea ultimului element
 - (d) Inserarea la mijloc
5. Intr-o lista simplu inlantuita, cum accesezi ultimul element?
 - (a) `list[last]`
 - (b) Traversezi lista de la `head`
 - (c) `head->prev`

- (d) `list.end()`
- 6. Care dintre urmatoarele operatii este mai eficienta pentru lista simplu inlantuita, comparativ cu vectorul dinamic?
 - (a) Accesarea elementelor pe baza de indice
 - (b) Inserarea la sfarsit
 - (c) Inserarea la inceput
 - (d) Sortarea
- 7. Care este complexitatea timp pentru distrugerea unei liste inlantuite?
 - (a) $O(1)$
 - (b) $O(n)$
 - (c) $O(\log_2 n)$
 - (d) $O(n * \log_2 n)$
- 8. Care dintre urmatoarele afirmatii despre listele simplu inlantuite cu alocare dinamica nu este adevarata?
 - (a) Poti insera la inceput in $O(1)$
 - (b) Poti accesa ultimul element in $O(1)$
 - (c) Nodurile sunt alocate dinamic
 - (d) Lista trebuie traversata pentru a gasi un anumit element
- 9. Stiind ca TAD-ul **Stack** respecta interfata pentru stiva prezentata la curs, ce se va afisa in urma executiei urmatorului subalgoritm?

```
s <- Stack()
s.push(4)
s.push(10)
x <- s.pop()
s.push(8)
y <- s.pop()
s.push(6)
{@print x}
```

- (a) 4
- (b) 10
- (c) 8
- (d) 6
- 10. Stiind ca TAD-ul **Set** respecta interfata pentru multime prezentata la curs, care este dimensiunea multimei in urma executiei urmatorului subalgoritm?

```

s <- Set()
s.add(4)
s.add(10)
s.add(4)
s.remove(4)
s.add(6)

```

- (a) 0
 - (b) 1
 - (c) 2
 - (d) 3
11. Scrieti un algoritm pentru a verifica daca un sir de caractere ce contine doar caracterele '(' si ')' este valid. Acesta este valid daca parantezele sunt inchise in ordinea corecta si fiecare paranteza deschisa are o paranteza inchisa ca si corespondent ¹.

Exemple:

- Input: "()" → Output: True (valid)
- Input: "((()))" → Output: True (valid)
- Input: "(((" → Output: False (invalid)
- Input: ")((" → Output: False (invalid)

¹Hint Folositi o stiva